A Repository-Based System for Research and Publication

Architecture, evidence and iteration in a Lean research repository

Will Cook*

5 October 2026

Abstract

Plectis is a repository-based research environment for retaining mathematical work across sessions and carrying selected results into publication. The public prototype brings questions, sources, proofs, computations and failed approaches together in eight problem collections. Its publication workflow records the formal-support status of paper statements and binds explanatory passages to cited sources, while keeping these records distinct. Researchers and agents choose the work; Python tools retrieve evidence, record Lean probes and regenerate views. Maintainers decide what to adopt, and a journal retains entered contributions and review decisions. A worked revision shows how an explanation can lose a hypothesis while its formal proofs remain valid. A historical author-run test rejected nine of ten false edits and accepted one false completion claim. Reader benefit, discovery rate and adoption by independent laboratories remain unmeasured.

1 Introduction

Research continues across sessions, contributors and drafts. The next researcher needs more than the latest result: which question remains open, why an approach failed, and what justifies the claims already published. Even a preserved formal proof leaves room for a later explanation to drop a hypothesis or describe an unresolved step as complete. A usable research record must retain these distinctions as the work changes.

Plectis supports this work in a public repository. Questions, prior sources, computations, proofs and unresolved steps remain available alongside proposed contributions and their review. The prototype is a local research environment: a researcher or agent chooses what to investigate and which tools to invoke. Broader autonomous discovery and services for other scientific fields remain proposals (Section 9).

The architecture separates authored records, derived views and adoption decisions (Figure 1). A successful proof probe can remain a session result; a revised explanation can remain a proposal. Acceptance updates the source records, after which builders regenerate their indexes and summaries.

Erdős Problems supplies the questions and their public discussion, while Lean and mathlib supply the formal language and library [1, 11, 76]. Formalisation blueprints link a written argument to named Lean declarations [35, 74], and the Equational Theories Project maintains shared problem records [75]. Prove2Me combines human-audited missions, source-linked milestones and separately submitted proofs, with intermediate results available for other agents to reuse [31, Sections 3–4].

Authorship and AI use. Will Cook built and directed the research infrastructure and reviewed the claims when he could. AI agents did most of the research and drafting. Cook did not independently verify every claim.

The question is how to keep a publication aligned with its changing research record. Assertions, formal supports, explanations and acceptance decisions need separate identities and links. Checks detect specified disagreements; preserving the mathematics remains a review task.

Section 2 describes the architecture; Sections 3–5 follow an explanation change through checking and review. Contributions, observations and related systems follow. Appendices A–D cover inspection, the worked proof, scope and attribution examples, and references grouped by purpose.

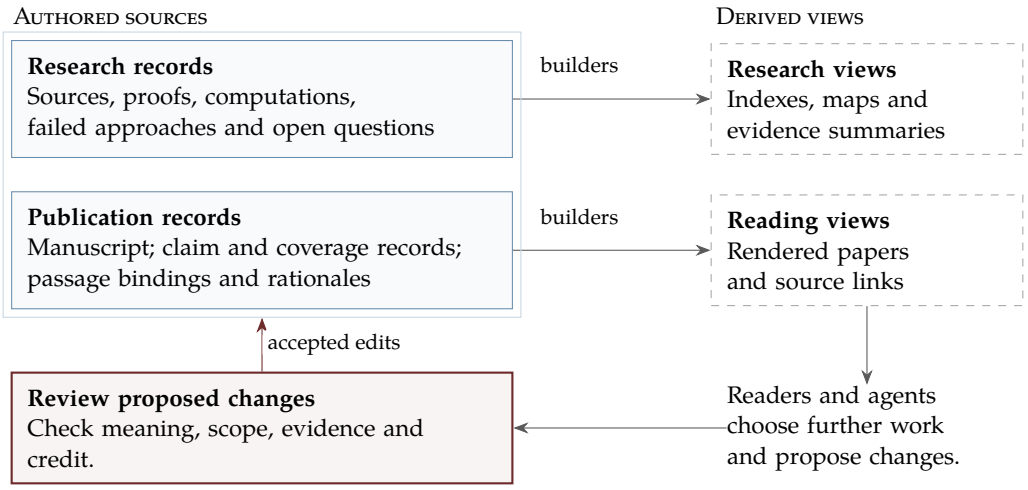


Figure 1: Authored records and derived views. Accepted edits change the relevant sources; builders regenerate views. Tool success alone does not adopt a claim.

2 A repository organised around problems

2.1 Stored records and derived views

Files store the research state: Lean and manuscript arguments, selected claims and obligations, evidence, entered returns and decisions. Indexes, coordinates and status summaries derive from them. Freshness checks detect disagreement; builders regenerate views. Neither adopts a claim. The `architecture guide` identifies each view's source, where repairs belong.

The coverage ledger follows a labelled *paper occurrence*, identified by location and statement digest, and lists its registered Lean supports and evidence status. A stronger formal result needs an authored relation note bound to both statements. The passage ledger records explanatory text, cited spans, evidence class and rationale. The first asks what supports the statement; the second exposes the explanation's sources for review. Neither generates prose from a proof.

2.2 Finding the relevant work

A problem collection retains proofs, counterexamples, exact experiments, corrected references and failed approaches, so later work can inspect them before choosing another route. Four views provide entry points without requiring a full Lean build (Figure 2).

A *declaration* is a named Lean definition, theorem or lemma. The declaration index locates it; the dependency index follows the declarations used by selected results. The interpretation graph adds authored accounts of declarations and their connections. The claim registry selects assertions for public reporting and records their status and remaining obligations.

For each problem: the question and literature, Lean statements and proofs, computations and certificates, claims and open obligations, failed approaches, and the paper with its supporting record.

View available without a Lean build	Question it answers
Declaration index	Where is the declaration? Source locations are exhaustive within the inventory.
Dependency index	Which declarations does this result use? Exact edges are available for selected starting declarations.
Interpretation graph	How has the work been interpreted? Authored accounts are selective and largely contextual, not individual proof reviews.
Claim registry	What is being claimed, with which status and open obligations? Only selected assertions have claim records.

Across the corpus: 8 problems, 1,903 Lean modules and 25 registered papers. The claim registry contains 160 claims in eight statuses, with 19 open obligations.

Figure 2: Contents of a problem collection and the four ways to inspect the recorded work. The counts refer to the whole corpus. Source locations, dependency edges, interpretations and claim statuses answer different questions.

A reader can start from a claim, locate its declarations, follow the recorded dependencies and request the source lines relevant to a question. Declaration locations cover the inventory; dependency edges are exact for selected starts. Interpretations are selective and largely contextual, not records of individual proof review. Lookup locates source without recording execution. A Lean check requires a focused build and inspection of its result.

2.3 Operations on the records

The entry points below act on the same repository records. They support a research iteration; their order of use is chosen by the researcher or agent.

Component	Input, output and responsibility
Task routing	`agent_entry.py` maps a stated task to instructions, relevant files and commands. It routes a request rather than choosing the research question.
Evidence retrieval	`query_corpus.py` returns problem, claim, declaration and paper-source records from the indexes.
Proof sessions	`proof_workbench.py` stores notes and submitted Lean probes, obtains verdicts from Lean, and binds a session claim to an accepted probe.
Statement evidence	`paper_evidence.py` locates each registered support at the recorded revision and checks its source and verification bindings; `paper_claim_evidence.py` derives status and gap reports from those records.
Editorial exchange	`short_paper_writer.py` audits manuscripts and packages proposed revisions without applying or approving the return.
Research history	`research_record.py` verifies the journal of entered returns and dispositions and reports the outstanding milestones of recorded outputs.
Derived views	`refresh_projections.py` runs builders in dependency order; `check_release.py` checks release contracts and freshness.

A separate self-audit, `systems_paper_evidence.py`, checks this systems manuscript's text and cited source bindings. These are implemented interfaces, not reports that every operation has been executed successfully.

An optional proof session illustrates the boundary between producing evidence and adopting a result. It stores notes, exact Lean inputs, process-derived receipts and a closing outcome. Its claim command requires an accepted probe and appends a session claim; it does not alter the reviewed public claim registry. The journal separately checks the sequence and hash chain of entered returns and dispositions. Later work can therefore inspect recorded inputs and decisions without the original conversation.

2.4 Responsibilities and adoption

A researcher chooses the question and the evidence needed to answer it. An agent can inspect files, run tools and prepare a change under the public instructions; a contributor can also send an ordinary argument or explanation without a clone or an agent. The maintainer reviews scope, attribution and unresolved obligations before adoption. These are responsibilities, not necessarily different people or agents: contributor and reviewer may coincide. The `agent quickstart` and `contribution guide` describe setup and review.

An iteration retrieves relevant work, investigates a question and returns an argument, experiment, correction or manuscript change with its evidence and limits. Review can adopt, revise, defer or reject the proposal. Accepted changes update the authored records; builders regenerate dependent views and release checks test consistency. Structured returns explicitly entered into the journal retain their dispositions. Ordinary issues and pull requests can reach review without a journal session (Section 6).

3 Worked case: reviewing an explanation

The worked case comes from Erdős Problem 257, which asks whether every infinite subseries of $1 + 1/3 + 1/7 + 1/15 + \cdots$ is irrational. The theorem used here proves irrationality under a weighted summability condition on the retained exponents, with the same conclusion for every infinite subset and every integer base at least two. Dropping that condition would claim the unresolved general case. Appendix B introduces the series, its antecedents and the proof.

Consider a contributor revising that explanation. The coverage row `res:weighted-support` in `paper_lean_coverage.json` fixes the printed statement and lists two Lean supports. The explanation here has a separate `passage record`, citing the ordinary proof and a formal declaration. The claim-evidence builder derives status or gap reports; the passage checker tests text and source bindings.

Neither Lean proof need change when the explanation loses its weighted hypothesis. The prose edit makes its passage binding stale; rebinding can remove that warning without repairing the claim. Review must compare the wording with the cited statements. Accepted edits update the explanation and its record before publication views are regenerated. Figure 3 compares the three versions. This is an illustration, not a new test run.

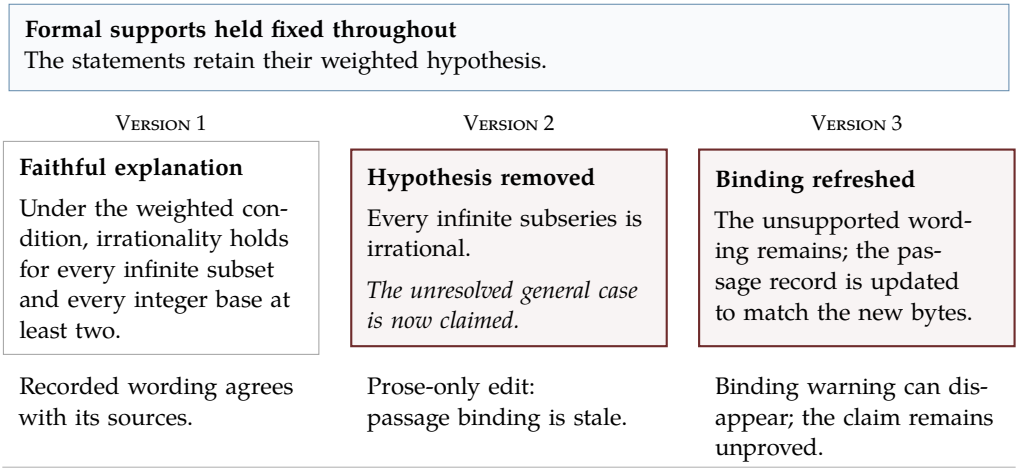


Figure 3: Hypothesis loss with unchanged formal supports. Rebinding can restore byte consistency while leaving the overclaim intact. This is an illustration, not a new test or historical edit 8.

4 Proof and publication checks

The two formal supports in the example overlap (Figure 7). One gives fixed-base irrationality and the all-base conclusion for every infinite subset; the other states the fixed-base hereditary conclusion explicitly. The *registered support set* is the list attached to this paper occurrence, not the dependency closure of those proofs. The publication audit checks the full registered list even when one declaration suffices for a particular clause. That checks the record; it adds no mathematical hypothesis. The resolver follows registered named inputs and checks source revisions. A named input is an additional assumed result and must remain visible. By contrast, the weighted condition in the example is a hypothesis inside the proved statement. Revising either the statement or its source requires checking the corresponding records again; evidence for one revision cannot silently support another.

Figure 4 separates the objects checked mechanically from the questions that remain for review.

Lean verifies that a proof establishes the formal statement written in the source; it does not verify whether that statement captures the intended mathematics or whether the paper describes it well. Comparator adds a separately stated challenge and a check of its selected Lean implementation under the permitted axioms [19, 20].

Each marked passage in this systems paper has a text record, specified source lines, an evidence class and a rationale. A passage may contain several sentences or a figure; it is a unit of source checking, not a grammatical category.

For the appendix's conclusion about infinite subsets, the sources explain why restriction preserves the weighted condition. The passage checker compares cryptographic hashes of the marked text and declared sources, rejects missing records or unbound passages, and requires a nonempty rationale. It does not assess whether the rationale justifies the text.

A matching digest establishes agreement with recorded bytes. It does not establish that the source entails the passage. In the worked revision, review must check the hypothesis and quantifiers even if both Lean links still resolve. Historical edit 8 tested a different failure: an unresolved step was described as complete (Table 1).

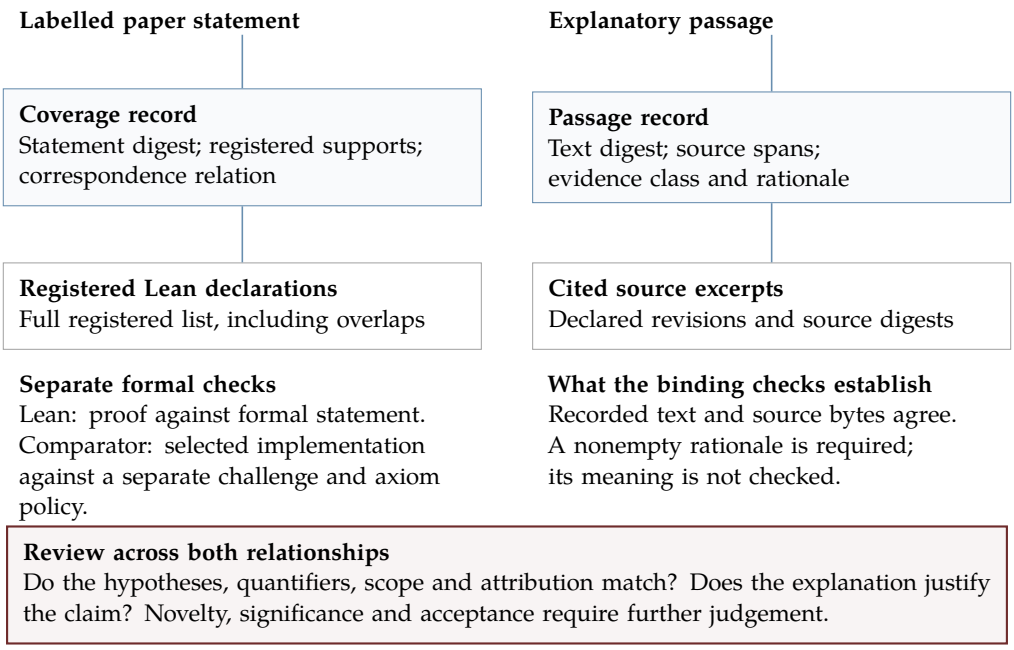


Figure 4: Two correspondence records. Lines are recorded links, not implications. Binding and formal checks do not establish the paper's intended meaning. Review responsibilities do not imply independent review.

Execution is a separate question. A matching declaration type and reachable source link do not identify a successful continuous-integration run for that module. The probe interface takes verdicts from the specified Lean process; callers cannot submit their own verdict through it. An actor with filesystem access can still alter the implementation or stored record. The protocol distinguishes contributor and reviewer roles but does not technically force a second independent mathematician. No independent human mathematical review of the corpus is recorded.

5 Publication and revision

Publication selects from the research record and explains the selected result. The short paper states its conditions, makes the decisive step intelligible and directs the reader to the full argument. A dependency graph can locate the ingredients; it cannot choose their explanatory order. That choice requires reading the argument. The companion retains the complete proof and subsidiary cases. Correspondence links take a short-paper claim to the passage that supports it. The assembler inlines authored companion sections into a flat manuscript with their cross-references; it supplies no missing proof step. The reading guide invites readers to try an example, reconstruct a step or vary a hypothesis. The reader chooses the amount of help; an explaining agent is instructed to respect the requested depth. These authoring choices are not measurements of understanding.

5.1 Permissions and tasks

An integrating agent may change the live checkout; an advisory agent returns a proposal against supplied sources. The repository calls these permissions Type A and Type B.

Permission to edit does not determine the task. Either assignment can concern research, proof or claim audit, literature and attribution, or exposition.

The expected product differs: a candidate argument, a source-located finding, an attribution proposal or a revised manuscript. Each needs the corresponding review.

Exposition preserves established hypotheses, conclusions and evidence status. A suspected mathematical defect is returned for a separate audit or research assignment.

5.2 From frozen inputs to an accepted revision

The writer first identifies the result, its antecedents, the difficult explanatory step and the unresolved questions, with source locations.

Selection remains an authorial decision; the manuscript checker tests source consistency and declared rules.

The exchange freezes selected manuscripts, required inputs, coverage records and audit rules under a manifest of their file digests. Writer and integrating reviewer then work from identifiable starting bytes, including any selected uncommitted changes.

An advisory model may work from that packet; a tool-enabled agent may prepare a proposal in a checkout. The packet identifies editable manuscript paths and read-only evidence and tools. Both routes return proposed changes for the same review of mathematical scope, wording and credit.

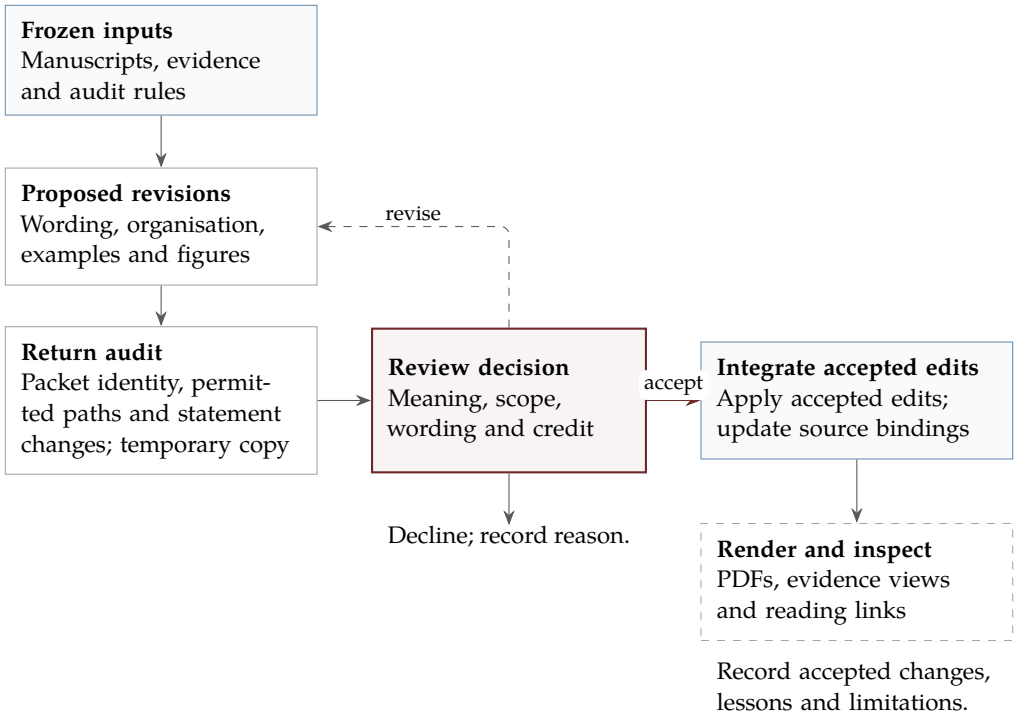


Figure 5: Only accepted edits enter live sources. The return audit neither applies edits nor executes returned programs. Review may request revision or decline; one agent may perform several responsibilities.

The return checker verifies packet identity and permitted paths, accounts for statement changes, and checks retained labels, citation keys and bibliography entries. It audits the proposed manuscript in a temporary copy of the frozen sources, leaving the live repository unchanged.

It neither applies changes nor executes returned programs. A declared statement change remains an explicit review obligation.

The reviewer then checks the whole statement, its hypotheses and implication directions against all registered supports. An ordinary argument keeps its actual evidence class wherever formalisation is incomplete.

After acceptance, the integrating reviewer edits the owning sources and updates affected evidence bindings. For an assembled companion, the authored sections are edited before rebuilding the flat paper. Integration then regenerates evidence and navigation and includes inspection of the compiled pages, figures and proof links.

5.3 Retaining improvements

A reusable writing lesson is a separate proposal, with an example and limits. Adopting it requires a reviewed change to the writing contract.

The packet and audit stages in Figure 5 are implemented by `short_paper_writer.py` and `paper_refinement.py`. A return can change the paper without changing the rules for later writers. Accepted lessons revise documents and instructions, not model weights; the procedure itself measures neither reader benefit nor autonomous discovery.

6 An open route for contributions

The same review boundary applies to contributions beyond exposition. An idea or reference needs no clone; a patch identifies the public commit on which it was prepared. An executable return includes changed files, commands and results, resource use, surviving limitations, and credit for the people and tools involved. Figure 6 follows both entry routes through triage and adoption. The protocol asks reviewers to reproduce executable evidence at the submitted base before reconciling it with current main, and to credit a substantive integration repair separately.

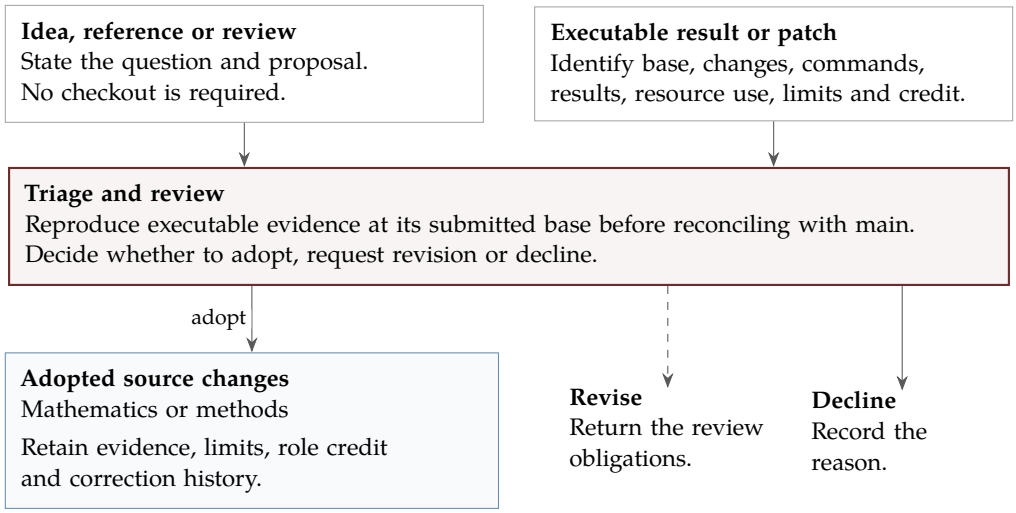


Figure 6: Two entry routes and three dispositions. Adopted work includes mathematics and methods. Only explicitly entered returns become journal events; adoption establishes neither independent review nor wider acceptance.

For explicitly entered returns, the journal preserves an append-only, hash-linked sequence of dispositions. A later correction can withdraw an earlier endorsement without erasing that history. Ordinary issues and pull requests need not become journal events. Research investigates the question; stewardship compares a return with prior work, groups related results, reconciles affected papers and identifies what to investigate next. The researcher or agent chooses that next

task. The public journal records entered decisions and does not schedule research. To decide what was learned, a later reviewer must compare the return with earlier work and inspect its evidence; a recorded disposition alone cannot supply that assessment.

The repository's proposed route from a local result to an upstream library includes a literature and library comparison, a proof of the extension and a second useful instance before expert submission [100]. This route is not automated, and the supplied record reports no Mathlib contribution from it. Credit distinguishes an idea from its implementation, formalisation and exposition. CRediT supplies a vocabulary for roles, not an authorship decision [104]. An attributed dependency or correction can record later use; it implies neither a scalar impact score nor a financial contract. Opening another problem collection still requires assembling a sourced question, explicit status, inspection routes and a review owner.

7 Recorded observations

7.1 Inventory and formal-support status

The frozen inventory contains 689 paper occurrences, of which 641 have exact or specialising Lean support; 23 depend on a named input and 25 have no recorded Lean support. Of the 641 exact-or-specialising occurrences, 616 are recorded as compared and 25 as queued. These are occurrence counts, not counts of distinct theorems or independently reviewed proofs. Exact support matches the printed statement; specialising support yields it by specialization. Named-input entries retain an additional assumed result. An entry without Lean support may have an ordinary proof; neither of these last two groups is counted as fully covered.

7.2 Historical false-edit trial

The historical publication-evidence record reports that nine of the ten deliberately false edits were rejected and one escaped. The edits were authored by the checker's author, the original run logs were not retained, and the other nine edits were not rerun against the extended checklist. The edits were applied separately, with the baseline restored between trials. The escaped edit changed a description of an unresolved step from "does not supply" to "completes", and the publication checks accepted it.

The follow-up tested the intact baseline and only the escaped edit against an added check. It supplies no post-repair ten-of-ten result. In that recorded follow-up, the post-repair witness accepts the current README and rejects a test copy containing the false clause. The trial uses one corpus and the checker's own selection of errors, with no manual-review control or ordinary continuous-integration comparator. The study locates a coverage boundary. Its nine rejections do not estimate how reliable the checker is.

7.3 Local integration and unperformed reader evaluation

On 23 September 2026, a maintainer-operated agent accepted one architecture handoff into a local integration history after focused checks of return routing and architecture packaging. The contributor and reviewer were the same agent. The receipt records no public pull request, human review or independent outside clone replay.

A proposed reader study would hold the mathematical task fixed and compare an ordinary repository, an information-equivalent static briefing and the navigation tools. It would measure understanding and severe errors alongside preparation, use, review and repair costs, retaining missing returns, timeouts and disagreements. No independent writing comparison, blind grading run or cold-reader experiment is reported, so there is no comparative reader result. The records above document inventory, a checker failure and one local integration; general reliability, reader understanding and mathematical transfer remain unmeasured.

Edit	Change	First detector	Outcome
edit 1	Conditional result labelled proved	Projection freshness	rejected
edit 2	Open-boundary clause deleted	Boundary wording	rejected
edit 3	Declaration coordinate moved	Source coordinates	rejected
edit 4	Paper source link retargeted	Source coordinates	rejected
edit 5	Claim assigned to two families	Registry structure	rejected
edit 6	Module count hand-edited	Projection freshness	rejected
edit 7	`native_decide` introduced	Proof-trust policy	rejected
edit 8	“does not supply" changed to “completes"	None	**escaped**
edit 9	Open-problem title mangled	Paper anchoring	rejected
edit 10	Orientation byte budget exceeded	Byte budget	rejected

Historical trial: edit 8
The false completion clause passed all recorded checks.

Later witness: baseline and edit 8
Baseline accepted, edit 8 rejected; the other nine edits were not rerun.

Table 1: The ten historical edits and their recorded first detectors; “proof-trust policy” refers to this repository’s permitted proof mechanisms, not to a general unsoundness claim about `native_decide`.

8 Related work

The closest comparisons concern the objects a system records and the work it lets contributors perform. Lean and mathlib provide formal statements, proofs and a maintained library; blueprints connect declarations to exposition [11, 76, 35]. Prove2Me provides a more direct comparison for agent-assisted collaboration. Appendix D retains the wider source map.

Prove2Me. We use its published version 2, dated 31 August 2026; the first version appeared on 28 August 2026 [31]. Its basic unit is an immutable theorem statement, stored separately from the proofs submitted for it. A proof is checked against the target type in a pinned verification environment. A *proof-sketch* derives a target using imported theorem statements, some of which may remain open. Contributors can prove those children separately; the target becomes verified when the required children are verified. Immutability makes those local proofs composable without changing their targets [31, Sections 3.1–3.2 and 4.2].

Human review is part of Prove2Me's design. A mission fixes a human-audited core of goals, definitions and milestones; each milestone pairs a source statement with its attested canonical formalisation. For read-back, an independent agent receives the Lean declaration and dependent definitions without the original source; a human compares its mathematical rendering with that source. Natural-language accounts accompany statements and proofs, statement descriptions support library search, and imports give contributors reuse credit. The discussion record also includes a disproved statement followed by a corrected replacement [31, Sections 3.3 and 4.3–4.5]. Explanation, reuse, correction and shared agent work are therefore established parts of this comparison.

For correspondence with a source, the closest analogue is Prove2Me's milestone: both designs identify the mathematical statement whose formalisation is being reviewed. Our coverage record follows a particular paper occurrence and can list several overlapping formal supports, with a relation note explaining their connection to the printed claim. It is not a proof-sketch: listing those supports does not compose proof terms or discharge open lemmas. A separate passage

record binds the explanation to sources and rationale, while entered contribution records retain revision decisions and role credit. The focus here is the consistency and review of these publication records across revisions. This difference of focus establishes neither priority for persistent research records nor a measured advantage over Prove2Me.

Polymath, BOINC and GIMPS supply precedents for small mathematical contributions and donated compute [99, 97, 98], while the Carleson blueprint shows formalisation organised into publicly claimable tasks [87].

The regeneration tools address the familiar build-system problem of keeping derived outputs consistent with their inputs [48]. They run an explicit ordered list of builders and then check the resulting views; this is not a general incremental scheduler. Source bindings and the journal add provenance for assertions and entered returns at particular revisions. Together these mechanisms make dependencies and changes available for inspection. Their effect on research efficiency and semantic correctness needs separate evaluation.

9 Limits

The prototype establishes a set of implemented record and review operations, with the limited observations in Section 7. It reports no comparative formalisation result or autonomous-optimisation experiment. Transfer to unseen mathematics and understanding by independent readers require direct evaluation. In particular, no controlled comparison shows that the contribution process converts compute into useful mathematics more effectively or increases discovery rate. There is also no evidence of inexpensive adoption by a new project or an advantage over an agent using a good README. Full scans and initial builds still have costs, and an open corpus cannot rule out prior model exposure to its contents.

Governance remains maintainer-centred. Review capacity is scarce and credit is contestable; openness alone does not equalise resources or resolve appeals. Repository adoption cannot confer acceptance by the wider mathematical community. Returned code is untrusted, and expensive or privileged continuous-integration jobs must not execute fork code with repository secrets [105]. A volunteer-compute service and extensions to physical science remain proposals requiring their own validation, resource and safety governance.

The Advisory Group on Mathematics and Artificial Intelligence at IAS recommends distinguishing papers fully understood by a responsible mathematician from AI output that its human prompters do not understand, and asks AI laboratories to support subsequent community-led understanding [10]. Its 29 September 2026 statement gives concrete release expectations for attribution, exposition, formalisation and disclosure of the research process.

Several of these proposed release practices have counterparts in the records described here [10, Section 2.B, Step I]: the short paper and companion paper explain and attribute the argument, the paper-to-Lean ledger records formal correspondence and its limits, and the files prepared for formal comparison identify its inputs. Those records make the result inspectable; they do not certify a mathematician's understanding. The architecture alone does not establish complete per-result model, prompt, time and cost disclosure, an account of campaign-wide selection and unsuccessful attempts, or an independent scholarly deposit. The further recommendations on funding community-led understanding and equitable model access address AI laboratories; this prototype does not demonstrate those institutional outcomes.

The contribution is an inspectable record from which to recover a claim's conditions, argument and limits, then submit a correction or extension against identified sources. Checks preserve specified links; review decides what the work justifies. Revision requires both.

A Reading the materials and reproducing checks

The materials support reading a result, inspecting its records and rerunning checks. Reading requires no Lean installation; proof replay requires the recorded sources and matching Lean environment. The routes below separate document checks from reproduced proofs.

A.1 Read a result and follow its sources

The reading guide routes readers through one problem or across the papers, including *Reading Eight Erdős Problems Together* [17]. Read a result's short paper, then follow its argument, attribution and formal-support references. Agents can use agent instructions and the workbench's optional proof sessions. Corrections follow the contribution guide and credit policy.

A.2 Inspect the records and rerun document checks

The experiment record contains the ten-edit matrix and limited follow-up in Section 4. The publication-check specification identifies required checks. These structured files trace evidence; they do not replace the paper's explanation.

From a complete public checkout, run:

```
python3 scripts/systems_paper_evidence.py
python3 scripts/build_systems_paper_counts.py
```

The first checks passage-to-source links and content hashes; the second compares the generated inventory with its records. Success establishes agreement, not the correctness of an explanation.

For proof replay, the Lean entry file and CI workflow identify build entry points. The claim register lists curated assertions; release checks cover wider publication requirements. Use the result's recorded revisions and environment.

A writing packet contains manuscripts, selected records and inspection tools, but no Lean tree. It supports document checks, not a full Lean, Comparator or release rerun.

A.3 Locate the formal supports of the worked example

For the weighted criterion in Appendix B, the paper-to-Lean coverage record identifies two formal supports: a weighted criterion and a version whose conclusion explicitly covers every infinite subset. Figure 7 shows the recorded links. The exact declaration names are provided for readers locating them in Lean.[1] These entries support the stated conclusion; they do not assert line-by-line agreement between the ordinary proof and the Lean proofs.

The recorded Comparator runs are the first comparison run and the second comparison run. Comparator checks a formal result against a separately stated challenge. These records concern the specified formal statements, not the historical attribution or the correctness of this paper's explanation.

The reciprocal-summability criterion attributed to Erdős in Appendix B has its own formal proof, also recorded in the second comparison run.[2]

Both runs are recorded as compared. Palomar entries are prepared, not submitted; comparison does not establish registry acceptance.

[1] Both are in namespace `ErdosProblems.Erdos257.PaperCompleteR8`: `divisibilityWeightedClaim` and `finitePrimeWeighted_fixedBase_hereditary`. The coverage row is `res:weighted-support`; the weighted comparison entries are `PalomarCorpus/E257` and `E257bc`.

[2] The declaration is `Erdos249257.irrational_erdosSupportSeries_of_summable_reciprocal`; its comparison entry is `PalomarCorpus/E257av`, recorded as compared.

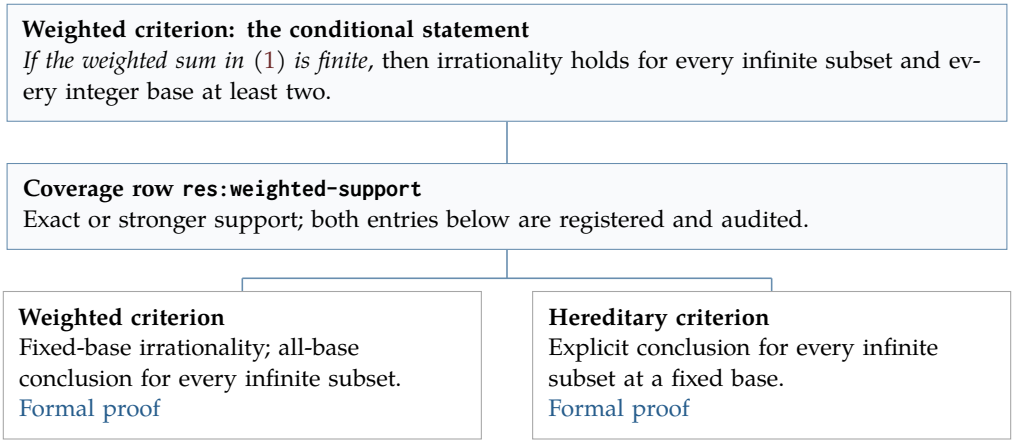


Figure 7: Registered supports have overlapping roles; lines are not proof dependencies. Auditing the full list adds no hypothesis to the weighted statement. Recorded comparison does not establish registry acceptance.

A.4 Interpret the recorded inventory

The recorded inventory counts occurrences of paper statements, rather than distinct new theorems. Its 689 entries comprise 499 with exact-or-stronger formal support, 142 obtained by specialising a formal result, 23 linked to a named formal ingredient only, and 25 with no registered Lean support. The last two categories do not certify the whole paper statement.

The contribution journal records 7 rounds and 25 returned contributions at the recorded journal revision.[3] Of the rounds, 2 are sealed, meaning no further arrivals are expected. Of the returns, 18 have been admitted for integration and 7 remain unreviewed; admission does not itself record integration. Earlier private-history rounds do not constitute a complete public journal.

Explanatory text is recorded for 139,818 of the 152,387 authored theorem-like declarations (91.8%). Most of these descriptions provide surrounding context. This inventory measures the presence of recorded interpretations, not individual proof review or demonstrated reader understanding.

A.5 Identify the version and the earlier design accounts

Repository links in this paper identify fixed file versions, so that a reader can inspect the material cited even if the working repository later changes. Unless a link states otherwise, its revision is `992cf60a27ef`; the Comparator runs identify their own inputs. No new Lean or Comparator run was performed for this revision.

For archival identification, the initial unified manuscript was prepared from the frozen packet at `ced96e58b337`, a local integration commit rather than public main. The 29 September state report lists it under the earlier title *Problem-Sized Lean Worlds*.

This paper supersedes the two earlier design accounts described below. Cite a mathematical result by the paper and version actually used; cite repository software using its software citation record at the chosen commit or release tag.

From a Cold Clone to a Proof Receipt (August 2026) describes the declaration, dependency, interpretation and claim views and the optional recorded proof session. Its counts and audit figures

[3] Journal revision: `d3ad85269386`.

belong to that snapshot; Section 2 describes the current task entry and inspected implementation.

From Spare Compute to Cumulative Mathematics (September 2026) describes contribution roles, reproduction against the original inputs, subsequent integration and role credit. Section 6 gives the current procedure. The earlier paper's volunteer-compute services, inexpensive adoption by new laboratories and increased discovery rate remain proposals or unmeasured outcomes. These manuscripts explain the design's history; they are not additional current specifications.

B The mathematical example: a criterion for irrationality

Section 3 used a theorem whose explanation could become false by losing a hypothesis. Following the Problem 257 paper [2], this appendix introduces the series and earlier work, states the weighted criterion, gives an example of its reach, and explains the averaging proof. Each part identifies information that a faithful summary must retain. The formal-support records are in Appendix A; the calculation is optional for readers interested only in the system.

B.1 The question and its antecedents

Begin with the series

$$1 + \frac{1}{3} + \frac{1}{7} + \frac{1}{15} + \cdots = \sum_{a=1}^{\infty} \frac{1}{2^a - 1}.$$

An infinite subseries retains the terms whose exponents belong to an infinite set A of positive integers. We call A the *support*: it specifies which terms are present. To allow other integer bases $b \geq 2$, write

$$X_A(b) = \sum_{a \in A} \frac{1}{b^a - 1}.$$

These series converge by comparison with a geometric series. The question concerns their arithmetic value: can an infinite choice of retained terms have a rational sum?

Problem 257 asks whether $X_A(2)$ is irrational for every infinite support A. The weighted theorem below gives a sufficient condition on A; it does not settle that unrestricted question [2, 1].

Erdős proved the all-base irrationality conclusion for pairwise coprime exponents satisfying $\sum_{a \in A} 1/a < \infty$. He also stated that coprimality could be removed, without supplying the details [3, pp. 222, 226]. The Problem 257 paper gives a direct proof of that stated extension, as well as the weighted criterion used here [2].

Duverney and Tachiya provide a methodological antecedent: they choose an arithmetic progression by congruences and use averaging to control the remaining contribution [4, Section 2, (2.3)–(2.9)]. The Problem 257 paper credits that selection method. Its weighted proof uses an additional average over progression lengths, which recovers a factor needed to sum the error bounds. The sources thus supply a classical criterion, an averaging method and the weighted extension, respectively.

B.2 A weighted condition on the retained exponents

The weighted condition gives a smaller contribution to exponents divisible by large powers of a fixed set of primes. For $a = 2^k m$ with m odd, it replaces the reciprocal weight $1/(2^k m)$ by $1/[m(2^{2^k} - 1)]$. More generally, for a finite nonempty set P of primes, the *P-part* of a is $h_P(a) = \prod_{p \in P} p^{v_p(a)}$, where $v_p(a)$ is the exponent of p in the prime factorisation of a. Thus $h_{\{2\}}(2^k m) = 2^k$.

The weighted-support theorem of the Problem 257 paper states that, if

$$W_{2,P}(A) := \sum_{a \in A} \frac{h_P(a)}{a(2^{h_P(a)} - 1)} < \infty, \tag{1}$$

then $X_B(b)$ is irrational for every integer $b \geq 2$ and every infinite $B \subseteq A$ [2, Theorem 1.2].

The conclusion is *hereditary*: it applies to every infinite subset of A. For a fixed base b, write $W_{b,P}(A)$ for the sum in (1) with 2 replaced by b. The fixed-base version assumes $W_{b,P}(A) < \infty$ and gives the same hereditary conclusion at that base. We prove this version below and recover all integer bases from the base-two hypothesis.

B.3 Why the weighted condition reaches further

To see why the weight matters, compare it with the older condition $\sum_{a \in A} 1/a < \infty$. Both are tests on the exponents; neither is needed merely for convergence of $X_A(b)$. The source paper gives

$$A_\star = \{2^k m : k \geq 1,\ m \text{ odd},\ 1 \leq m \leq 2^{2^k}\}, \qquad P = \{2\}.$$

This set is arranged in disjoint layers according to the power 2^k dividing an exponent. Within layer k, reciprocal summation discounts each odd m by 2^{-k}; weighted summation discounts it by $(2^{2^k} - 1)^{-1}$. The second discount is much smaller.

Write $S_r = \sum_{1 \leq m \leq 2^r,\ m \text{ odd}} 1/m$ for the sum of odd reciprocals up to 2^r; layer k uses $r = 2^k$. Grouping these integers into intervals $[2^j, 2^{j+1})$ gives $r/4 \leq S_r \leq r$ for integers $r \geq 2$.

The reciprocal contribution of each disjoint layer is at least $2^{-k} S_{2^k} \geq 1/4$. Its weighted contribution satisfies

$$\frac{S_{2^k}}{2^{2^k} - 1} \leq \frac{2^k}{2^{2^k} - 1} \leq 2^{1-k},$$

so the layer contributions have a convergent geometric majorant in the weighted sum.

Thus $\sum_{a \in A_\star} 1/a$ diverges, whereas $W_{2,\{2\}}(A_\star)$ converges. The weighted criterion therefore applies to some supports excluded by the reciprocal-summability condition. This is the source paper's ordinary calculation, with no separate entry in the paper-to-Lean ledger [2, calculation following the weighted proof].

B.4 The proof by two averages

Fix an integer base $b \geq 2$, an infinite support A and a finite nonempty set P with $W_{b,P}(A) < \infty$. The proof has two parts. Rationality would force a certain positive remainder to be at least the reciprocal of a fixed denominator. Averaging will make that remainder smaller. The main difficulty is that one average leaves an error bound that cannot be summed using the hypothesis; a second average over powers of two supplies the missing factor.

The lower bound under rationality. Suppose that $X_A(b) = p/q$ with integers p, q and $q > 0$. For a positive integer N, multiplication by $b^N - 1$ separates an integer contribution from each summand, leaving

$$\Delta_{b,A}(N) = \sum_{a \in A} \frac{b^{N \bmod a} - 1}{b^a - 1} = (b^N - 1)X_A(b) - J_{b,A}(N), \qquad N \geq 1, \tag{2}$$

where $J_{b,A}(N) = \sum_{a \in A,\, a \leq N} \sum_{j=1}^{\lfloor N/a \rfloor} b^{N-ja}$ is an integer. Here $N \bmod a$ is the remainder on dividing N by a. The symbol Δ denotes the positive remainder after this particular integer is subtracted, not necessarily a fractional part in $[0, 1)$.

Every summand is nonnegative, and one is positive because the infinite set A contains an exponent larger than N. Hence $q\Delta_{b,A}(N)$ is a positive integer, and $\Delta_{b,A}(N) \geq 1/q$.

We obtain a contradiction by finding a multiple $N = tQ$ with smaller remainder. Divisibility helps because the summand indexed by a vanishes whenever $a \mid Q$.

Choose a finite nonempty $F \subset A$ outside which the weighted sum is less than ε, and make Q a multiple of every element of F. Those terms then disappear for every t; we must control the remaining terms on average.

Averaging over a residue period. With $b = 2$, $Q = 4$ and $a = 6$, the residues $tQ \bmod a$ are $4, 2, 0$, and the corresponding mean contribution to Δ is

$$\frac{1}{3}\left(\frac{15}{63} + \frac{3}{63} + 0\right) = \frac{2}{21} \leq \frac{1}{9}.$$

For a general exponent a, put $g = \gcd(a, Q)$. The residues run through the multiples of g in a cycle of length a/g. Replacing each numerator $b^r - 1$ by b^r bounds the sum over one cycle by $1/(b^g - 1)$. There are at most $T/(a/g)$ complete cycles and one unfinished cycle among the first T terms. Dividing their total by T gives

$$\frac{1}{T}\sum_{t=1}^{T} \frac{b^{tQ \bmod a} - 1}{b^a - 1} \leq \underbrace{\frac{g}{a(b^g - 1)}}_{\text{complete cycles}} + \underbrace{\frac{1}{T(b^g - 1)}}_{\text{unfinished cycle}}. \tag{3}$$

The first term has the shape of the weighted hypothesis once suitable prime powers have been put into Q. The second has lost the factor $1/a$. Summability of the first therefore does not justify summing the unfinished-cycle bounds over all exponents. This is the difficulty that a single progression average leaves unresolved.

Averaging over dyadic lengths. The source proof averages over the M lengths $T = 2^j$, $M \leq j < 2M$; these powers-of-two lengths are called dyadic. Thus M counts the lengths being averaged, whereas T counts the terms in one progression average. For nonnegative weights α_a indexed by positive integers, with $\sum_a \alpha_a/a < \infty$, it uses

$$\sum_{j=M}^{2M-1} 2^{-j} \sum_{a \leq Q2^j} \alpha_a \leq 2Q \sum_a \frac{\alpha_a}{a}. \tag{4}$$

Indeed, a fixed a enters only when $2^j \geq a/Q$, and the sum of the admissible 2^{-j} is at most $2Q/a$. This recovers the missing factor before summing over a. Dividing by the number M of lengths then reduces the cost from $2Q$ to $2Q/M$.

Choosing the modulus and the averaging lengths. The modulus must cancel F while making the remaining exponents amenable to one of two estimates. Fix a positive common multiple L of F, put $p_* = \max P$, and, for $H \ge 2p_*$, set

$$Q = L \prod_{p \in P} p^{\lfloor \log_p H \rfloor}, \qquad G = \lfloor H/p_* \rfloor.$$

If $h_P(a) \le H$, then $h_P(a) \mid Q$. Since $n/(b^n - 1)$ decreases for positive integers n, the complete-cycle term in (3) is bounded by $h_P(a)/[a(b^{h_P(a)} - 1)]$. Its sum outside F is less than ε.

If $h_P(a) > H$, then $\gcd(a, Q) \ge G$: either every P-prime-power component of $h_P(a)$ is at most H, so $h_P(a) \mid Q$, or one exceeds H, and its truncated component in Q is greater than $H/p_* \ge G$. Summing (3) over these exponents up to QT costs at most

$$\frac{G(1 + \log(QT)) + Q}{b^G - 1} :$$

the harmonic sum supplies $1 + \log(QT)$, and at most QT unfinished-cycle terms supply Q.

Increasing M therefore has two effects. It reduces the first group's unfinished-cycle cost Q/M, but increases the second group's cost through $\log T < 2M \log 2$. We need both $Q/M \to 0$ and $GM/b^G \to 0$, not merely a long average. With F and L fixed, $Q \le LH^{|P|}$ and $G = H/p_* + O(1)$. The choice $M = \lfloor b^{G/2} \rfloor$ meets both requirements.

The source proof combines these estimates into the finite mean

$$\begin{aligned} &\frac{1}{M} \sum_{j=M}^{2M-1} \frac{1}{2^j} \sum_{t=1}^{2^j} \Delta_{b,A}(tQ) \\ &\qquad \le \varepsilon + \frac{2QW_{b,P}(A)}{M} + \frac{G(1 + \log Q + 2M \log 2) + Q}{b^G - 1} + 4\,2^{-M}. \end{aligned} \tag{5}$$

The terms bound, respectively, the weighted tail over complete cycles, its unfinished cycles, the exponents with large gcd, and the exponents beyond QT. The second term uses (4) with $\alpha_a = \mathbf{1}_A(a)/(b^{h_P(a)} - 1)$; the last uses the geometric tail bound $4/T$. After ε, F and L have been fixed, every term after ε tends to zero as $H \to \infty$ [2, Section 2, weighted-support proof].

The left side of (5) is an average with nonnegative weights summing to one: each length has weight $1/M$, distributed equally among its 2^j terms. For large H, at least one displacement is therefore less than 2ε. Taking $0 < \varepsilon < 1/(2q)$ contradicts the lower bound $1/q$. This argument gives no rate of decay in N. Passing to an infinite $B \subseteq A$ only decreases the weighted sum, and $b^{h_P(a)} - 1 \ge 2^{h_P(a)} - 1$ gives the all-base conclusion from the binary hypothesis [2, Section 2].

This is the information the worked explanation must preserve: the restriction on the support, the distinction from earlier criteria, and the reason for the second average. The formal-support links identify proofs of the stated conclusion. They do not by themselves explain these choices or prevent a summary from dropping the hypothesis.

C Attribution and scope in three further examples

The following cases distinguish a result's exact scope, its dependence on earlier work and its formal verification. They illustrate further information a faithful account must preserve, rather than steps in the preceding proof or additional systems experiments.

C.1 Counting repeated terms changes the question

A running least common multiple is the least common multiple of the terms seen so far in an increasing sequence. Its value can repeat. For integers with prime factors only 2, 3 and 5, the source paper's example begins $1, 2, 6, 12, 60, 60, 120, 360, 360, \ldots$ [28]. A reciprocal sum counting every occurrence contains $1/60$ twice; one counting distinct values contains it once.

The Problem 269 record proves a result for distinct running least common multiples. The original question counts them with multiplicity, so the result does not automatically answer it [28].

The same record credits Fan's earlier public two-prime result despite an independently reported derivation, and separately identifies the relevant Hecke–Mahler series literature [28, 29, 26, 27]. Independent derivation does not remove the obligation to cite prior work.

C.2 An explicit description and a stronger antecedent

The Euler totient $\varphi(n)$ counts the integers from 1 to n coprime to n. The Problem 249 paper studies its subsequences $\varphi(k^j n + r)$, where $k \geq 2$ is fixed, $j \geq 0$ and $0 \leq r < k^j$. For each level j, it describes the integer linear relations and gives an explicit basis [13]. Coons's non-k-regularity theorem says that no finite set generates these subsequences over all levels. Martin's stronger separation theorem already implies the independence needed for the basis; the explicit reductions and integral relations are a different contribution. The direct proof uses the Chinese remainder theorem and Dirichlet's theorem on primes in arithmetic progressions, following an argument in Yazdani that Yazdani credits to Shallit [14, 15, 16].

Reorganising a proof, making its consequences explicit and formalising it are distinguishable contributions. None makes an inherited ingredient new.

C.3 A construction, its formalisation and the historical question

Problem 1041 concerns connecting roots of a polynomial by short curves in a strict lemniscate: for a complex polynomial f, this is the region $|f(z)| < 1$. The degree-seven counterexample construction is due to the forum contributor ani [23, 21]. The recorded Lean result refutes the exact statement in the Formal Conjectures repository; independent human review of its correspondence with the curve-length wording in the 1958 paper has not been recorded [77, 24, 22]. The construction, its formalisation and that historical correspondence are separate claims.

Readers inspecting the formal result can follow the counterexample assembly and the Hausdorff-length proofs.[4]

The coverage row records exact-or-stronger Lean support for these statements.

Comparator remains pending for this result, so its recorded Lean support should not be described as a completed formal comparison.

The other seven motivating targets remain unresolved in this work.

A revised statement must retain its scope, attribution and registered supports. Sharpening, merging or moving it requires renewed review of its relation to every support and named input.

[4] The four declarations, all in namespace `Erdos1041.Counterexample`, are `erdos1041_counterexample`, `erdos1041_counterexample_hausdorff`, `erdos1041_hausdorff_negation` and `erdos1041_hausdorff_answer_false`.

D A reading map for the wider literature

Section 8 compares the closest systems; Appendices B and C explain the mathematical antecedents used in the examples. Figure 8 is a guide to the remaining references and to sources cited by the two historical design accounts in Appendix A. Its groups indicate why a source is cited, not that every listed method is implemented here or that every work has been evaluated against this system.

Area	Reason for citation
Formal foundations	Lean and mathlib; library maintenance and contributor practice [11, 76, 92, 100]
Source and statement checks	Comparator, kernel replay, vacuity, linters and property tests [19, 20, 52, 43, 51]
Formal-to-prose tools	Blueprints, source documentation, translation and refactoring [35, 74, 37, 36, 61, 70]
Retrieval and interaction	Premises, proof states and exact declaration neighbourhoods [39, 88, 90, 91, 89, 93]
Proof automation	Premise selection and tactic search [49, 50, 59]
Library structure	Declaration networks and repository knowledge graphs [45, 46, 95]
Assumptions and reuse	Proof mining, generalisation, lemma mining and theory exploration [41, 42, 53, 54]
Growing proof knowledge	Library learning, evolving procedures and candidate statements [55, 56, 57, 58, 62, 83]
Research agents	Executable objectives, proof cost and evolving tasks [78, 79, 30, 25, 60]
Agent collaboration	Bounties, planner/worker state, retrieval and compiler feedback [80, 81, 82, 31, 44, 75]
Evaluation	Research budgets, proof engineering and documentation tasks [84, 71, 72, 73, 106, 102, 103]
Validity and intended meaning	Research case studies, interactive proving and public accounting [18, 32, 40, 33]
Public infrastructure	Problem sources, formal conjectures, verification and collaboration [1, 77, 101, 87, 99, 97, 98, 104, 105]
Writing and maintenance	Mathematical exposition, build dependencies and provenance [85, 69, 48, 47, 94]
Understanding and reuse	Explanation, mathematical understanding, reuse and research possibilities [5, 6, 9, 86, 7, 63, 96]
Different community views	Declarations, dissent, benchmarks, research infrastructure and pace [8, 64, 65, 66, 34, 67, 68]
Mathematical antecedents	Weighted support, totient ranks and arithmetic separation [2, 3, 4, 13, 14, 15, 16]
Other mathematical records	Running LCM, lemniscates and cross-problem connections [28, 29, 26, 27, 12, 38, 22, 21, 23, 24, 17]

Figure 8: Reading map for this paper and its two historical predecessors. The groups record uses of sources, not a performance ranking or a list of implemented components. Appendices B and C distinguish mathematical antecedents, refinements and independent rediscovery.

References

[1] T. F. Bloom, *Erdős problems*, erdosproblems.com, accessed September 2026.

[2] W. Cook, *Weighted and Covered Supports for Mersenne Subseries*, short paper on Erdős Problem 257, September 2026, PDF.

[3] P. Erdős, *On the irrationality of certain series*, Math. Student 36 (1968), 222–226 (issued 1969), scan.

[4] D. Duverney and Y. Tachiya, *Refinement of the Chowla–Erdős method and linear independence of certain Lambert series*, Forum Math. 31 (2019), 1557–1566, DOI.

[5] B. Kra, *Deep theorems were scarce and difficult and so became an effective mechanism to identify deep thought. AI has broken this system*, guest post on *What's new*, 13 September 2026, blog post.

[6] H. Cohn, *The technical debt of AI-generated mathematics*, guest post on *What's new*, 15 September 2026, blog post.

[7] T. Tao, thread on mining open problems, Mathstodon, 8 September 2026, thread.

[8] Twenty-five Fields Medallists, *A severe misalignment of AI in mathematics*, declaration posted on *What's new*, 11 September 2026, blog post.

[9] G. Sanderson, *If math is more than proof, we need to better celebrate the rest of it*, guest post on *What's new*, 18 September 2026, blog post.

[10] Advisory Group on Mathematics and Artificial Intelligence at IAS, *Responsible Release of AI-Generated Mathematics*, 29 September 2026, recommendations, PDF.

[11] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, in *Automated Deduction, CADE 28*, Lecture Notes in Computer Science 12699, 2021, pp. 625–635, DOI.

[12] P. Erdős and R. L. Graham, *Old and New Problems and Results in Combinatorial Number Theory*, Monographies de L'Enseignement Mathématique 28, 1980, p. 61.

[13] W. Cook, *Integral Relations among Totient Sections*, short paper on Erdős Problem 249, revised 30 September 2026, PDF.

[14] M. Coons, *(Non)Automaticity of number theoretic functions*, J. Théor. Nombres Bordeaux 22 (2010), 339–352, DOI, Theorem 3.2.

[15] G. Martin, *Simultaneous inequalities among values of the Euler phi-function*, 2006, arXiv:math/0603053v1, Theorem 1.

[16] S. Yazdani, *Multiplicative functions and k-automatic sequences*, J. Théor. Nombres Bordeaux 13 (2001), 651–658, Numdam.

[17] W. Cook, *Reading Eight Erdős Problems Together*, synthesis paper, September 2026, source, including the periodic-chain theorem.

[18] T. Feng, T. Trinh, G. Bingham, et al., *Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems*, 2026, arXiv.

[19] Lean FRO, *Comparator*, 2025, GitHub.

[20] ammkrn, *nanoda_lib*, an independent type checker for Lean 4, GitHub.

[21] T. F. Bloom, *Erdős Problem #1041*, erdosproblems.com, accessed September 2026.

[22] P. Erdős, F. Herzog and G. Piranian, *Metric properties of polynomials*, J. Analyse Math. 6 (1958), 125–148, DOI.

[23] ani, post in the Problem 1041 discussion thread, 7 September 2026, erdosproblems.com forum.

[24] Formal Conjectures, pull request 6505, *Erdős 1041: mark solved with answer(False) and link a formal proof*, merged 23 September 2026, GitHub.

[25] T. Zheng, X. Wu, Z. Zhang, et al., *Dream-RSI: Recursive Self-Improvement through Evolving Worlds*, 2026, arXiv:2609.14858.

[26] J. H. Loxton and A. J. van der Poorten, *Arithmetic properties of certain functions in several variables III*, Bull. Austral. Math. Soc. **16** (1977), 15–47.

[27] Y. Bugeaud and M. Laurent, *Transcendence and continued fraction expansion of values of Hecke–Mahler series*, Acta Arith. **209** (2023), 59–90.

[28] W. Cook, *Distinct running least common multiples*, short paper on Erdős Problem 269, September 2026, PDF.

[29] S. Fan, comment on Erdős Problem #269, erdosproblems.com forum, 26 June 2026, forum post.

[30] N. Patel, A. Rammal, A. Hayat, R. Munos, and J. Kempe, *Learning to Discover Interesting Mathematics*, arXiv:2609.28603v1, 2026, arXiv:2609.28603v1.

[31] S. Chen, K. Marwaha, X. Lu, H. Yuen, and T. Peng, *Prove2Me: An Open Collaborative Platform for Scaling Math Formalization*, 2026, arXiv.

[32] T. Feng, T. H. Trinh, G. Bingham, et al., *Towards Autonomous Mathematics Research*, 2026, arXiv.

[33] T. Tao, *AI contributions to Erdős problems*, GitHub, accessed September 2026.

[34] T. Ringer, *Becoming a benchmark*, guest post on *What's new*, 17 September 2026, blog post.

[35] P. Massot, *leanblueprint*, plasTeX plugin for Lean formalisation blueprints, 2020, software repository.

[36] Y. Zhang, Y. Sun, T. Suzuki, J. D. Lee, and F. Liu, *LeanMarathon: Toward Reliable AI Co-Mathematicians through Long-Horizon Lean Autoformalization*, 2026, arXiv:2606.05400.

[37] S. Hattori, T. Matsuzaki, and M. Fujiwara, *Natural Language Translation of Formal Proofs through Informalization of Proof Steps and Recursive Summarization along Proof Structure*, in *Proceedings of the 18th International Natural Language Generation Conference*, 2025, pp. 376–389, ACL Anthology.

[38] S. Akiyama and H. Kaneko, *Multiplicative analogue of Markoff–Lagrange spectrum and Pisot numbers*, Adv. Math. 380 (2021), 107547, p. 3, reporting the theorem of A. Dubickas, J. Number Theory 117 (2006), 222–239, arXiv:1911.06170v6.

[39] K. Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, NeurIPS 2023, Datasets and Benchmarks Track, arXiv:2306.15626.

[40] C. Li, Z. Lai, D. An, J. Hu, and Z. Wen, *Advancing Mathematical Research via Human-AI Interactive Theorem Proving*, 2025, arXiv:2512.09443v2.

[41] U. Kohlenbach, *Applied Proof Theory: Proof Interpretations and their Use in Mathematics*, Springer Monographs in Mathematics, Springer, Berlin, 2008.

[42] A. Gandhi, A. R. Tadipatri, and T. Gowers, *Automatically Generalizing Proofs and Statements*, in *16th International Conference on Interactive Theorem Proving*, LIPIcs 352, 2025, pp. 12:1–12:18, DOI.

[43] Lean community, *Batteries*, the `unusedArguments` linter in `Batteries/Tactic/Lint/Misc.lean`, GitHub, accessed September 2026.

[44] J. Xin, A. Schneidman, C. Cummins, K. Ram, S. Ganesh, and J. Limperg, *AXLE: A Cloud Infrastructure for Lean 4 Theorem Proving Utilities*, 2026, arXiv:2606.26442.

[45] X. Li, N. Peng, S. Severini, and P. Shafto, *The Network Structure of Mathlib*, 2026, arXiv:2604.24797.

[46] F. Huch, *Structure in Theorem Proving: Analyzing and Improving the Isabelle Archive of Formal Proofs*, extended abstract, 2022, arXiv:2209.13305.

[47] J. de Kleer, *An assumption-based TMS*, Artificial Intelligence 28 (1986), 127–162, DOI.

[48] A. Mokhov, N. Mitchell, and S. Peyton Jones, *Build Systems à la Carte*, Proc. ACM Program. Lang. 2 (ICFP), 2018, article 79, DOI.

[49] T. Zhu, J. Clune, J. Avigad, A. Q. Jiang, and S. Welleck, *Premise Selection for a Lean Hammer*, 2025, revised 2026, arXiv:2506.07477.

[50] J. Limperg and A. H. From, *Aesop: White-Box Best-First Proof Search for Lean*, in *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 2023, pp. 253–266, DOI.

[51] Lean community, *Plausible*, a property testing framework for Lean 4, GitHub, accessed September 2026.

[52] H. Chockler, A. Gurfinkel, and O. Strichman, *Beyond Vacuity: Towards the Strongest Passing Formula*, Form. Methods Syst. Des. 43 (2013), 552–571, DOI.

[53] C. Kaliszyk and J. Urban, *Lemma Mining over HOL Light*, LPAR-19, LNCS 8312, 2013, pp. 503–517, DOI.

[54] M. Johansson, D. Rosén, N. Smallbone, and K. Claessen, *Hipster: Integrating Theory Exploration in a Proof Assistant*, CICM 2014, LNCS 8543, pp. 108–122, DOI.

[55] K. Ellis et al., *DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake-Sleep Bayesian Program Learning*, 2020, arXiv:2006.08381.

[56] H. Wang, H. Xin, et al., *LEGO-Prover: Neural Theorem Proving with Growing Libraries*, ICLR 2024, arXiv:2310.00656.

[57] Y. Zhang, J. Sun, et al., *DreamProver: Evolving Transferable Lemma Libraries via a Wake-Sleep Theorem-Proving Agent*, 2026, arXiv:2604.26311.

[58] W. Ye, Z. Guan, et al., *ProofEvolve: Neuro-Symbolic Evolution for Formal Automated Theorem Proving*, 2026, arXiv:2608.26334.

[59] S. Böhme and T. Nipkow, *Sledgehammer: Judgement Day*, IJCAR 2010, LNCS 6173, pp. 107–121, DOI.

[60] I. Berlot-Attwell, T. Sesterhenn, F. Rudzicz, and X. Si, *Is This LLM Library Learning? Evaluation Must Account For Compute and Behaviour*, EACL 2026, pp. 3534–3568, DOI.

[61] Y. Fu, P. Liu, Z. Wang, and K. Yuan, *Proof-Refactor: Refactoring Generated Formal Proofs into Modular Artifacts*, 2026, arXiv:2606.03743.

[62] N. Onda et al., *LeanConjecturer: Automatic Generation of Mathematical Conjectures for Theorem Proving*, 2025, arXiv:2506.22005.

[63] D. Litt, *A beginning for mathematics*, 13 September 2026, blog post.

[64] T. Chu, *The AI dissenter viewpoint*, *Proofs and Prompts*, 9 August 2026, blog post.

[65] B. Green and forty-one other Fellows and Foreign Members of the Royal Society, *Open letter to Sir Paul Nurse, President of the Royal Society*, *Proofs and Prompts*, 17 September 2026, blog post.

[66] W. T. Gowers, *Why I didn't sign the Fields medallists' letter*, 17 September 2026, blog post.

[67] D. Koukoulopoulos, *A CERN for AI-assisted science?*, guest post on *What's new*, 17 September 2026, blog post.

[68] B. Antieau, *Fast math/slow math*, 15 September 2026, blog post.

[69] D. E. Knuth, T. Larrabee and P. M. Roberts. *Mathematical Writing*. Stanford report CS1193, based on the autumn 1987 course, Section 1.

[70] Dayu Yang et al., *DocAgent: A Multi-Agent System for Automated Code Documentation Generation*, arXiv:2504.08725v3, 2025, Sections 2–3.

[71] Xinchen Wang, Ruida Hu, Cuiyun Gao, Pengfei Gao, and Chao Peng. *Evaluating Repository-level Software Documentation via Question Answering and Feature-Driven Development*. arXiv:2604.06793v1, 2026.

[72] Huajian Xin, Luming Li, Xiaoran Jin, Jacques Fleuriot, and Wenda Li. *APE-Bench: Evaluating Automated Proof Engineering for Formal Math Libraries*. arXiv:2504.19110v3, 2026.

[73] Auguste Poiroux, Gail Weiss, Viktor Kunčak, and Antoine Bosselut. *Reliable Evaluation and Benchmarks for Statement Autoformalization*. arXiv:2406.07222v3, 2025.

[74] T. Zhu, P. Monticone, J. Avigad and S. Welleck. *LeanArchitect: Automating Blueprint Generation for Humans and AI*. arXiv:2601.22554v1, 2026, Methods and Case Studies.

[75] M. Bolan et al. *The Equational Theories Project: Advancing Collaborative Mathematical Research at Scale*. arXiv:2512.07087v2, 2025, Project Management and Data Management.

[76] The mathlib Community. *The Lean Mathematical Library*. CPP 2020. DOI: 10.1145/3372885.3373824.

[77] Google DeepMind. *Formal Conjectures*. Repository README, consulted 29 September 2026; descriptive context only.

[78] A. Novikov et al. *AlphaEvolve: A coding agent for scientific and algorithmic discovery*. arXiv:2506.13131v1, 2025, Task Specification and Evolution.

[79] B. Georgiev, J. Gómez-Serrano, T. Tao and A. Z. Wagner. *Mathematical exploration and discovery at scale*. arXiv:2511.02864v3, 2025.

[80] C. E. Brown, C. Kaliszyk and J. Urban. *Agent Hunt: Bounty Based Collaborative Autoformalization With LLM Agents*. arXiv:2603.06737v1, 2026.

[81] M. Kripner and M. Straka. *OpenProver: Agentic and Interactive Theorem Proving with Lean 4*. arXiv:2607.09217v1, 2026, State and Memory Management.

[82] D. Ma et al. *OProver: A Unified Framework for Agentic Formal Theorem Proving*. arXiv:2605.17283v1, 2026, Introduction.

[83] Z. Yang et al. *CircuitProver: Agentic Lean 4 Theorem Proving with Reusable Circuit Proof Library for Hardware Verification*. arXiv:2607.27259v1, 2026.

[84] H. Wijk et al. *RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts*. arXiv:2411.15114v1, 2024; official abstract used here.

[85] P. R. Halmos. *How to Write Mathematics*. L'Enseignement Mathématique 16 (1970), 123–152; sections on organising a mathematical presentation.

[86] Various authors. *Writings on AI and Mathematics*. Supplied anthology, `AI_ESSAYS_SOURCE.pdf`, snapshot 22 September 2026; Kra, pp. 31–33; Riehl, pp. 39–40; Cohn, pp. 41–44; Gowers, pp. 55–60; Sanderson, pp. 73–77. Riehl's essay is also available as Why I do mathematical research.

[87] L. Becker et al., *A Blueprint for the Formalization of Carleson's Theorem on Convergence of Fourier Series*, 2025, arXiv:2405.06423.

[88] L. Aniva, C. Sun, B. Miranda, C. Barrett, and S. Koyejo, *Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4*, in *Tools and Algorithms for the Construction and Analysis of Systems*, 2025, pp. 116–137, DOI.

[89] S. Kurgan et al., *TheoremGraph: Bridging Formal and Informal Mathematics*, 2026, arXiv:2606.25363.

[90] J. Asher, *LeanExplore: A Search Engine for Lean 4 Declarations*, 2025, arXiv:2506.11085.

[91] G. Gao et al., *LeanSearch v2: Global Premise Retrieval for Lean 4 Theorem Proving*, 2026, arXiv:2605.13137.

[92] A. Baanen, M. R. Ballard, J. Commelin, B. Gin-ge Chen, M. Rothgang, and D. Testa, *Growing Mathlib: Maintenance of a Large Scale Mathematical Library*, in *Intelligent Computer Mathematics*, 2025, arXiv:2508.21593.

[93] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, arXiv:2604.16347.

[94] R. Souza, T. Poteet, B. Etz, D. Rosendo, A. Gueroudji, W. Shin, P. Balaprakash, and R. Ferreira da Silva, *LLM Agents for Interactive Workflow Provenance: Reference Architecture and Evaluation Methodology*, WORKS at the ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis, 2025, DOI.

[95] S. Abedu, S. Khatoonabadi, and E. Shihab, *Synergizing LLMs and Knowledge Graphs: A Novel Approach to Software Repository-Related Question Answering*, 2024, revised 2025, arXiv:2412.03815.

[96] T. Tao, *Mathematics in the age of AI*, 2026, arXiv:2608.16753.

[97] D. P. Anderson, *BOINC: A Platform for Volunteer Computing*, Journal of Grid Computing 18 (2020), 99–122, DOI.

[98] Great Internet Mersenne Prime Search, *GIMPS*, project documentation and discovery-credit record, mersenne.org, accessed August 2026.

[99] Polymath Project, *General polymath rules*, project rules, accessed August 2026.

[100] Lean community, *Contributing to mathlib*, contributor guide, accessed August 2026.

[101] Palomar Registry, *About Palomar* and *Contribution policy*, registry documentation and submission standard, accessed August 2026.

[102] J. Henkel, *The Mathematician's Assistant: Integrating AI into Research Practice*, 2025, arXiv:2508.20236.

[103] J. Dekoninck et al., *The Open Proof Corpus: A Large-Scale Study of LLM-Generated Mathematical Proofs*, 2025, arXiv:2506.21621.

[104] NISO, *CRediT: Contributor Roles Taxonomy*, role definitions, accessed August 2026.

[105] GitHub, *Preventing pwn requests*, GitHub Actions security guidance, documentation, accessed August 2026.

[106] M. Yuan et al., *End-to-End Testing of Open-Source Hardware Documentation Developed in Large Collaborations*, 2023, arXiv:2309.05942.